Quantum numerical linear algebra

Dong An

Beijing International Center for Mathematical Research,
Peking University

dongan@pku.edu.cn

April 11, 2026

Overview

Introduction to quantum computing

Basic linear algebra operations

Matrix functions

Application: Hamiltonian simulation

Application: Linear systems of equations

Summary

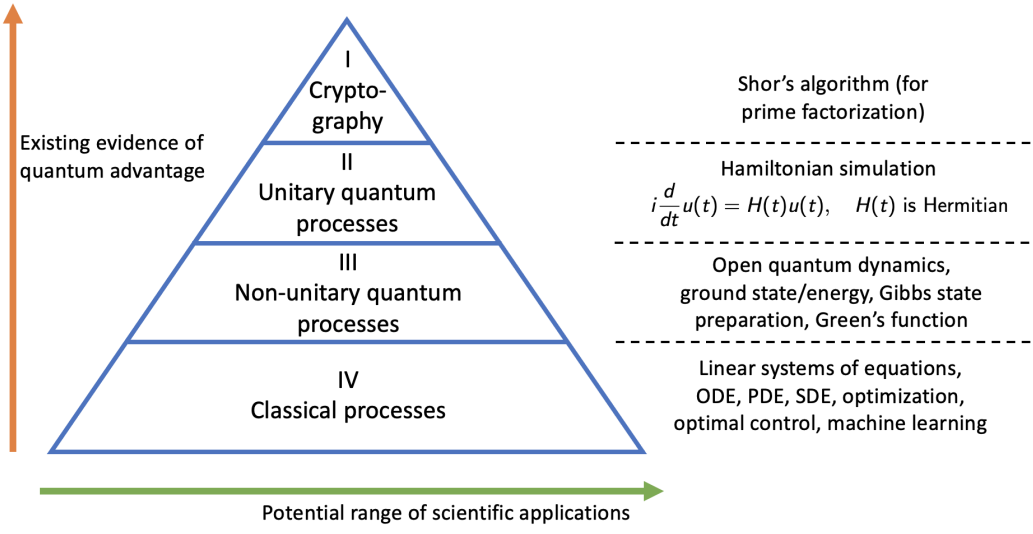
Introduction to quantum computing

Quantum computing

"If you want to make a simulation of nature, you'd better make it quantum mechanical, and by golly it's a wonderful problem, because it doesn't look so easy. "
[Feynman (1982)]

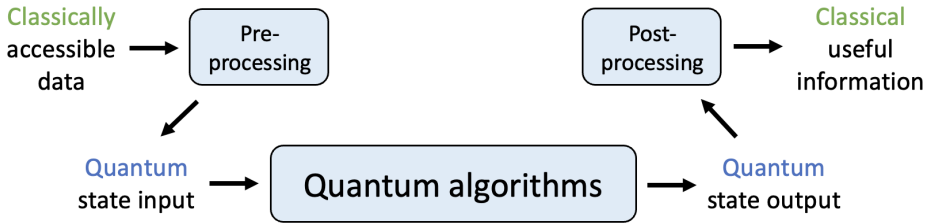
- Using the nature of quantum mechanics to perform computing
- Quantum advantage:
  - Unstructured search: Quantum cost $\mathcal{O}(\sqrt{N})$ v.s. Classical cost $\mathcal{O}(N)$
  - Fourier transform: Quantum cost $\mathcal{O}((\log N)^2)$ v.s. Classical cost $\mathcal{O}(N \log N)$

Quantum algorithm hierarchy[1]



[1] Adapted from [Lin Lin. Lecture Notes on Quantum Algorithms for Scientific Computation]

Quantum algorithm flow map



Input: quantum states

A qubit (quantum bit): $\mathbb{C}^2/\|\cdot\|$

- Instead of either 0 or 1, quantum allows superposition $\alpha|0\rangle + \beta|1\rangle$
- Measurement: we get 0 with probability $|\alpha|^2$, and 1 with probability $|\beta|^2$
- Orthonormal basis $|0\rangle = (1,0)^T$ and $|1\rangle = (0,1)^T$

Multi-qubit system: $\mathbb{C}^{2^n}/\|\cdot\|_2$, $N = 2^n$

- Superposition of 2^n possible states $|j\rangle$ $(0 \le j \le 2^n - 1)$

$$|u\rangle = \sum_{j=0}^{2^n-1} \alpha_j |j\rangle \cong (\alpha_0, \alpha_1, \cdots, \alpha_{2^n-1})^T$$

Tensor product states:

$$|u\rangle = |u_A\rangle \otimes |u_B\rangle = |u_A\rangle\,|u_B\rangle$$

Input: quantum states

Dirac notations:

- $|u\rangle = (\alpha_0, \alpha_1, \cdots, \alpha_{N-1})^T \in \mathbb{C}^N$, $\| |u\rangle \| = 1$
- Computational basis: $\{|j\rangle\}_{j=0}^{N-1}$

$$|u\rangle = \sum_{j=0}^{N-1} \alpha_j |j\rangle$$

- $\langle u| = |u\rangle^\dagger$
- Inner product: $\langle u|v\rangle$
- Observable: $\langle u|O|u\rangle$

Quantum algorithms

Quantum algorithms: Implement a series of unitary transformation

$$U_K \cdots U_2 U_1 \left|\psi_{\text{in}}\right\rangle = \left|\psi_{\text{out}}\right\rangle$$

- A quantum version of numerical linear algebra

Post-processing

$$|u\rangle = \sum_{j=0}^{N-1} \alpha_j |j\rangle$$

Measurement (under computational basis): collapse to $|j\rangle$ with probability $|\alpha_j|^2$

- Extract amplitude information or more general $\langle u|O|u\rangle$
- Post-select on "good subspace"

$$|\psi\rangle = \sqrt{p}\,|0\rangle\,|\psi_{\text{out}}\rangle + \sqrt{1-p}\,|\text{bad}\rangle$$

  - Each run of the algorithm is an independent trial with success probability p

    Expected number of repeats: $\mathcal{O}(1/p)$

  - Amplitude amplification (AA): first applies reflections to amplify the good part, then measure

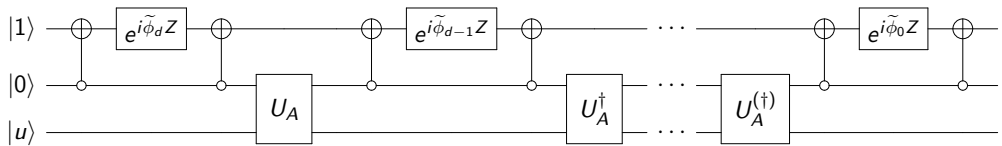
    Expected number of repeats with AA: $\mathcal{O}(1/\sqrt{p})$

Complexity and quantum circuits

Quantum complexity:

- Query complexity: number of oracle calls
- Circuit depth: number of coherent layers

Quantum circuits: graphical representation of quantum algorithms



Basic linear algebra operations

Input model: vectors

Input model: state preparation oracle

$$O_u : |0\rangle \to |u\rangle = \sum_{j=0}^{2^n-1} u_j |j\rangle$$

Constructing O_u is generally hard, but can be efficient in special cases[2]

[2] [arXiv:quant-ph/0208112], [arXiv:2201.11495]

Input model: matrices

Definition (Block-encoding)

Let A be a 2^n-by-2^n matrix. A block-encoding of A is a 2^{n+a}-by-2^{n+a} unitary U_A such that

$$A \approx \alpha \left(\langle 0|^{\otimes a} \otimes I \right) U_A \left(|0\rangle^{\otimes a} \otimes I \right),$$

or equivalently

$$U_A \approx \begin{pmatrix} \frac{1}{\alpha} A & * \\ * & * \end{pmatrix}.$$

- α is called the block-encoding factor and should satisfy $\alpha \geq \|A\|$
- Serve as both input and output models

Input model: matrices

$$U_A \approx \begin{pmatrix} \frac{1}{\alpha}A & * \\ * & * \end{pmatrix}.$$

- For an arbitrarily given matrix A, constructing U_A is in general hard
- Special cases: unitary, sparse matrices, structured matrices, $\cdots$ [3]

[3] [arXiv:1806.01838], [arXiv:2203.10236]

Matrix-vector multiplication

Input:

Block-encoding of A:

$$A \approx \alpha \left(\langle 0| \otimes I \right) U_A \left(|0\rangle \otimes I \right)$$

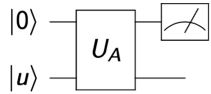
$$U_A \approx \begin{array}{c} \begin{array}{cc} \langle 0| & \langle 1| \end{array} \\ \begin{pmatrix} \frac{1}{\alpha} A & * \\ * & * \end{pmatrix} \end{array} \begin{array}{c} |0\rangle \\ |1\rangle \end{array}$$

or $$U_A \approx |0\rangle\langle 0| \otimes \frac{A}{\alpha} + |0\rangle\langle 1| \otimes * + |1\rangle\langle 0| \otimes * + |1\rangle\langle 1| \otimes *$$

Quantum state:

$$|u\rangle = \sum_{j=0}^{2^n-1} u_j |j\rangle$$

'Algorithm': applying block-encoding



or $$U_A |0\rangle |u\rangle \approx \frac{1}{\alpha} |0\rangle A |u\rangle + c |1\rangle |*\rangle$$

or $$\begin{pmatrix} \frac{1}{\alpha} A & * \\ * & * \end{pmatrix} \begin{pmatrix} u \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\alpha} Au \\ * \end{pmatrix}$$

Need to measure the first ancilla qubit

Success probability: $\left(\| A |u\rangle \| / \alpha \right)^2$

Number of repeats (after amplitude amplification): $\mathcal{O}\left(\alpha / \| A |u\rangle \| \right)$

Matrix addition: Linear combination of unitaries (LCU)

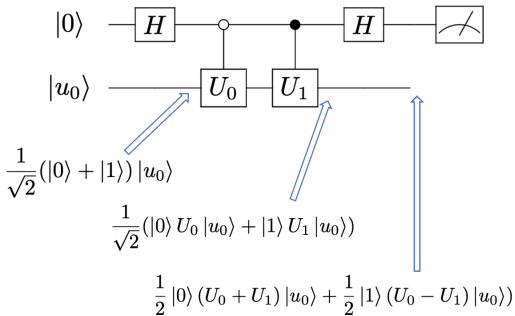
Task: given a set of unitary operators U_j and coefficients $c_j \in \mathbb{R}_+$, compute

$$\sum_j c_j U_j$$

- General matrix addition: U_j's are block-encodings
- Vector addition: U_j's are state preparation oracles

Matrix addition: LCU

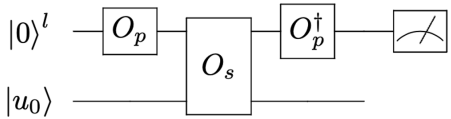
A toy example: computing $\frac{1}{2}(U_0 + U_1)\,|u_0\rangle$



$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

$$|0\rangle \to \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$
$$|1\rangle \to \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

General: computing $\sum_j c_j U_j$



Prepare Oracle $O_p : |0\rangle \to \frac{1}{\sqrt{\|c\|_1}} \sum_j \sqrt{c_j}\,|j\rangle$

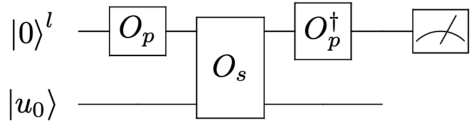
Select Oracle $O_s = \sum_j |j\rangle\langle j| \otimes U_j$

Matrix addition: LCU

$$
\begin{aligned}
|0\rangle |u_0\rangle &\xrightarrow{O_p} \frac{1}{\sqrt{\|c\|_1}} \sum_j \sqrt{c_j} |j\rangle |u_0\rangle \\
&\xrightarrow{O_s} \frac{1}{\sqrt{\|c\|_1}} \sum_j \sqrt{c_j} |j\rangle U_j |u_0\rangle \\
&\xrightarrow{O_p^\dagger} \frac{1}{\|c\|_1} |0\rangle \sum_j c_j U_j |u_0\rangle + |\perp\rangle
\end{aligned}
$$

- Repeats: $\mathcal{O}(\|c\|_1 / \| \sum c_j U_j |u_0\rangle \|)$
- Overall complexity depends on the cost of constructing O_p and O_s

General: computing $\sum_j c_j U_j$



Prepare Oracle $O_p : |0\rangle \to \frac{1}{\sqrt{\|c\|_1}} \sum_j \sqrt{c_j} |j\rangle$

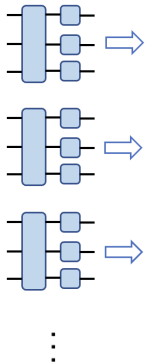
Select Oracle $O_s = \sum_j |j\rangle \langle j| \otimes U_j$

Hybrid implementation of LCU: Importance sampling

$$u = \sum_j c_j U_j |u_0\rangle \quad \Longrightarrow \quad u^\dagger O u = \sum_{j,j'} c_j c_{j'} \langle u_0 | U_j^\dagger O U_{j'} | u_0 \rangle .$$

Classical | Quantum | Classical

Sample (j, j') with probability $p \propto c_j c_{j'}$



$o_1 = \langle u_0 | U_{j_1}^\dagger O U_{j_1'} | u_0 \rangle$

$o_2 = \langle u_0 | U_{j_2}^\dagger O U_{j_2'} | u_0 \rangle$

$o_3 = \langle u_0 | U_{j_3}^\dagger O U_{j_3'} | u_0 \rangle$

$\vdots \qquad \vdots$

Average all the o's

Summary of basic linear algebra operations

Input models:

- Vector: quantum state
- Matrix: block-encoding

Algorithms:

- Matrix-vector multiplication
- Matrix/vector addition: LCU

Other algorithms (not covered today):

- Inner product/observable: SWAP test, Hadamard test
- Matrix multiplication

Reference: [Section 4, arXiv:1806.01838]

Matrix functions

Matrix functions

- For Hermitian matrices: eigenvalue transformation

$$A = V\mathrm{diag}(\lambda_j)V^\dagger \quad \rightarrow \quad f(A) = V\mathrm{diag}(f(\lambda_j))V^\dagger$$

- For general matrices: singular value transformation

$$\begin{aligned} A = W\mathrm{diag}(\sigma_j)V^\dagger \quad &\rightarrow \quad f(A) = W\mathrm{diag}(f(\sigma_j))V^\dagger \\ &\xrightarrow{\text{or}} \quad f(A) = V\mathrm{diag}(f(\sigma_j))V^\dagger \\ &\xrightarrow{\text{or}} \quad f(A) = W\mathrm{diag}(f(\sigma_j))W^\dagger \end{aligned}$$

Main result

Suppose that U_A is the block-encoding of a Hermitian matrix A with $\|A\| \leq 1$, and $p(x)$ is a real-coefficient polynomial such that

1. degree of $p(x)$ is d,
2. $|p(x)| \leq 1$ for all $x \in [-1, 1]$.

Then, $p(A)$ can be block-encoded with complexity

$$\mathcal{O}(d)$$

[3] Gilyen Etal [arXiv:1806.01838]

Applications

- Solving linear systems of equations: $f(x) = 1/(\kappa x)$
- Hamiltonian simulation: $f(x) = \exp(-ixt)$
- Filtering
- Amplitude amplification:

$$U|0\rangle|\psi\rangle = \frac{1}{q}|0\rangle A|\psi\rangle + |\perp\rangle \quad \rightarrow \quad \widetilde{U}|0\rangle|\psi\rangle = \frac{1}{2}|0\rangle A|\psi\rangle + |\perp\rangle$$

$$f(x) = qx/2, \quad x \in [-1/q, 1/q]$$

$$f(x) \approx p(x) \quad \text{where} \quad \deg(p(x)) \sim q\log(1/\epsilon)$$

- .....

Toy example: Chebyshev polynomials

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x),$$
$$T_k(\cos\theta) = \cos(k\theta)$$

Toy example: Chebyshev polynomials

Consider 2-by-2 matrices (let $\theta = \arccos \lambda$)

$$U(\lambda) = \begin{pmatrix} \lambda & \sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & -\lambda \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

$$O = U(\lambda)Z = \begin{pmatrix} \lambda & -\sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & \lambda \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}.$$

Then

$$O^k = \begin{pmatrix} \cos(k\theta) & -\sin(k\theta) \\ \sin(k\theta) & \cos(k\theta) \end{pmatrix} = \begin{pmatrix} T_k(\lambda) & * \\ * & * \end{pmatrix}$$

where $T_k = \cos(k\theta)$ is the Chebyshev polynomial.

- For Hermitian matrix case $T_k(A) = VT_k(\Lambda)V^\dagger$: for each eigenvalue, find its corresponding 2-dimensional subspace and perform this O^k

Qubitization

Suppose $A = \sum \lambda_j |v_j\rangle \langle v_j|$ and U_A is its *Hermitian* block-encoding ($U_A = U_A^\dagger$).

$$U_A |0\rangle |v_j\rangle = |0\rangle A |v_j\rangle + * = \lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\, |\perp_j\rangle$$

where $\Pi |\perp_j\rangle = 0$, $\Pi = |0\rangle \langle 0| \otimes I$.
Apply U_A again yields

$$U_A^2 |0\rangle |v_j\rangle = \lambda_j(\lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\, |\perp_j\rangle) + U_A \sqrt{1-\lambda_j^2}\, |\perp_j\rangle$$

$$U_A |\perp_j\rangle = \sqrt{1-\lambda^2}\, |0\rangle |v_j\rangle - \lambda_j |\perp_j\rangle$$

Invariant space: $\mathcal{H}_j = \text{span}\{|0\rangle |v_j\rangle, |\perp_j\rangle\}$. We may write

$$[U_A]_{\mathcal{H}_j} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}, \quad [\Pi]_{\mathcal{H}_j} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

[3] [arXiv:1610.06546]

Qubitization

$$[U_A]_{\mathcal{H}_j} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}, \quad [\Pi]_{\mathcal{H}_j} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

Let

$$Z_\Pi = 2\Pi - 1, \quad [Z_\Pi]_{\mathcal{H}_j} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Then

$$O = U_A Z_\Pi, \quad [O]_{\mathcal{H}_j} = \begin{pmatrix} \lambda_j & -\sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & \lambda_j \end{pmatrix},$$

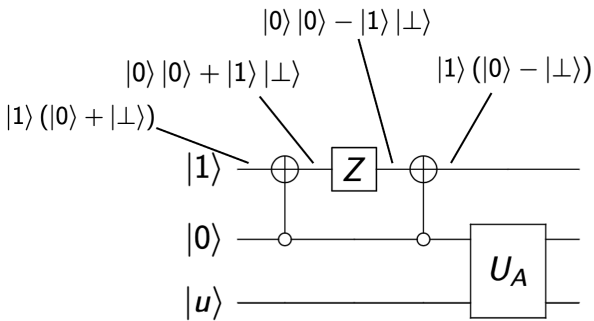
and thus

$$[O^k]_{\mathcal{H}_j} = \begin{pmatrix} T_k(\lambda_j) & * \\ * & * \end{pmatrix}, \quad O^k = \begin{pmatrix} T_k(A) & * \\ * & * \end{pmatrix}$$

Qubitization

$$U_A = \begin{pmatrix} A & * \\ * & * \end{pmatrix}, \quad U_A = U_A^\dagger, \quad \Pi = |0\rangle\langle 0| \otimes I, \quad Z_\Pi = 2\Pi - 1$$

$$(U_A Z_\Pi)^k = \begin{pmatrix} T_k(A) & * \\ * & * \end{pmatrix}$$

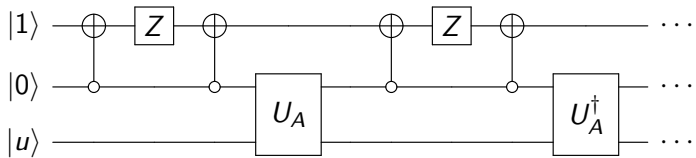


$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Qubitization

If U_A is not Hermitian (but A is still Hermitian): switching between two 2d subspaces

- Alternatively apply U_A and $U_A^\dagger$



Quantum signal processing (QSP)

Let us start with the 2-by-2 matrix again

$$U(\lambda) = \begin{pmatrix} \lambda & \sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & -\lambda \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad O(\lambda) = U(\lambda)Z$$

we have shown that

$$O(\lambda)^d = \begin{pmatrix} T_d(\lambda) & * \\ * & * \end{pmatrix}.$$

Consider a more general

$$U_\Phi(\lambda) = e^{i\phi_0 Z} O(\lambda) e^{i\phi_1 Z} O(\lambda) e^{i\phi_2 Z} \cdots O(\lambda) e^{i\phi_{d-1} Z} O(\lambda) e^{i\phi_d Z} = e^{i\phi_0 Z} \prod_{j=1}^{d} \left[O(\lambda) e^{i\phi_j Z} \right]$$

where $\Phi = (\phi_0, \phi_1, \cdots, \phi_d) \in \mathbb{R}^{d+1}$ are called phase factors

Quantum signal processing (QSP)

Theorem (QSP)

Let

$$O(\lambda) = \begin{pmatrix} \lambda & -\sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & \lambda \end{pmatrix}.$$

Then there exist phase factors $\Phi = (\phi_0, \phi_1, \cdots, \phi_d) \in \mathbb{R}^{d+1}$ *such that*

$$U_\Phi(\lambda) = e^{i\phi_0 Z} \prod_{j=1}^{d} \left[O(\lambda) e^{i\phi_j Z} \right] = \begin{pmatrix} P(\lambda) & -Q(\lambda)\sqrt{1-\lambda^2} \\ Q^*(\lambda)\sqrt{1-\lambda^2} & P^*(\lambda) \end{pmatrix}$$

if and only if $P(\lambda), Q(\lambda)$ *are complex-coefficient polynomials such that*

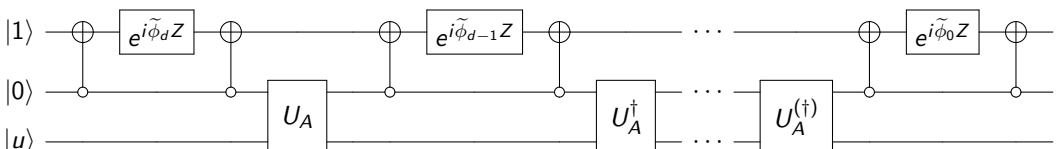
1. $deg(P) \leq d$, $deg(Q) \leq d-1$,
2. P *has parity* d *mod* 2 *and* Q *has parity* $d-1$ *mod* 2,
3. $|P(\lambda)|^2 + (1-\lambda^2)|Q(\lambda)|^2 = 1$ *for all* $\lambda \in [-1, 1]$.

[3] [arXiv:1606.02685]

QSP + Qubitization

High-dimensional matrix case is also valid through qubitization

Algorithm:



- Output: block-encoding of $P(A)$
- Query complexity: d

QSP for real polynomials

Theorem (QSP for real polynomials)

Let

$$O(\lambda) = \begin{pmatrix} \lambda & -\sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & \lambda \end{pmatrix}.$$

Given a real-coefficient polynomial $p(\lambda)$ such that

1. *$deg(p) = d$,*
2. *p has d mod 2 parity,*
3. *$|p(\lambda)| \leq 1, \forall\, \lambda \in [-1, 1]$.*

Then there exist two complex-coefficient polynomials $P(\lambda), Q(\lambda)$ and phase factors $(\phi_0, \phi_1, \cdots, \phi_d) \in \mathbb{R}^{d+1}$ such that $Re(P(\lambda)) = p(\lambda)$ and

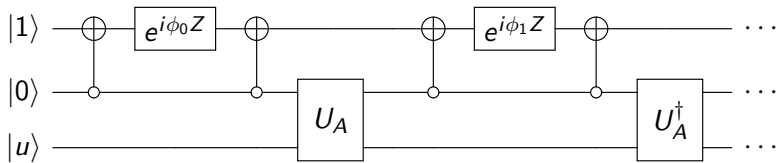
$$U_\Phi(\lambda) = e^{i\phi_0 Z} \prod_{j=1}^{d} \left[O(\lambda) e^{i\phi_j Z} \right] = \begin{pmatrix} P(\lambda) & -Q(\lambda)\sqrt{1-x^2} \\ Q^*(\lambda)\sqrt{1-x^2} & P^*(\lambda) \end{pmatrix}.$$

QSP, qubitization, and QSVT

Through qubitization, for

1. any Hermitian matrix A with $\|A\| \leq 1$ and its block-encoding U_A,
2. any d-degree real polynomial $p(\lambda)$ with $|p(\lambda)| \leq 1$ for all $\lambda \in [-1, 1]$,

we can block encode $p(A)$ with $\mathcal{O}(d)$ cost.



If A is not Hermitian, we are performing singular value transformation[4].

[4] Gilyen Etal [arXiv:1806.01838]

Phase factors

Finding phase factors was a hard task at the time when QSP was proposed, but has been practically solved so far.

- Direct methods:
  - Remez exchange algorithm
  - roots of polynomials [arXiv:1806.01838]
  - Capitalization [arXiv:2003.02831]
  - Prony's method [arXiv:2202.02671]
- Iterative methods:
  - optimization based algorithm [arXiv:2002.11649]
  - fixed point iteration [arXiv:2209.10162]
  - Nonlinear Fourier transform [arXiv:2505.12615]

QSP reference: [arXiv:2510.00443]

Application: Hamiltonian simulation

Hamiltonian simulation

$$\frac{d}{dt}u(t) = -iHu(t), \quad 0 \leq t \leq T, \quad H \text{ is Hermitian}$$

- Unitary transformation:

$$e^{-iHt}$$

- A natural (and very promising) application of quantum computers
- Hamiltonian simulation algorithms: Product formula, Quantum walk, Truncated Taylor series, QSVT, Randomization methods, $\cdots$

QSVT for Hamiltonian simulation

$$U(T) = e^{-iHT}$$

Consider the function

$$f(x) = e^{-ix\|H\|T} = \cos(\|H\|Tx) - i\sin(\|H\|Tx)$$

Polynomial approximation:

- Taylor expansion:

$$\cos(\|H\|Tx) \approx \sum_{k=0}^{\mathcal{O}(\|H\|T+\log(1/\epsilon))} \frac{(-1)^k \|H\|^{2k} T^{2k}}{(2k)!} x^{2k}$$

- Jacobi-Anger expansion:

$$\cos(Tx) \approx J_0(\|H\|T) + 2 \sum_{k=1}^{\mathcal{O}(\|H\|T+\log(1/\epsilon))} (-1)^k J_{2k}(\|H\|T)\, T_{2k}(x)$$

QSVT for Hamiltonian simulation

Algorithm: use QSVT to implement $\cos(TH)$ and $\sin(TH)$, then use LCU to linearly combine them

Query complexity:

$$\mathcal{O}\left(\|H\|T + \log\left(\frac{1}{\epsilon}\right)\right)$$

- Optimal scaling
- Drawback: cannot exploit structures inside H

$$H = \sum_{\gamma} H_{\gamma}, \quad H = -\Delta_x + V(x)$$

Product formula

$$H = A + B$$

Idea: split time evolution of H into product of evolutions of A and B

Lie-Trotter product formula:

$$e^{-iHT} = \lim_{L\to\infty} \left(e^{-iBT/L} e^{-iAT/L}\right)^L$$

- First-order product formula: $U_1(t) = e^{-iBt}e^{-iAt} \approx e^{-iHt} + \mathcal{O}(t^2)$
- Second-order product formula: $U_2(t) = e^{-iAt/2}e^{-iBt}e^{-iAt/2} \approx e^{-iHt} + \mathcal{O}(t^3)$
- Suzuki recursion:

$$U_{2p+2}(t) = U_{2p}(s_p t)^2 U_{2p}((1-4s_p)t) U_{2p}(s_p t)^2, \quad s_p = (4 - 4^{1/(2p+1)})^{-1}$$

Quantum complexity: commutator scaling, suboptimal in T and ϵ

$$\mathcal{O}\left(\frac{\alpha_{\text{comm}}^{1/p} T^{1+1/p}}{\epsilon^{1/p}}\right)$$

Multi-product formula

Motivation: simultaneously achieve commutator scaling and optimal time/precision dependence

Idea: extrapolation

$$U_{\mathrm{MPF}}(t) = \sum_{j=1}^{M} a_j U_2(t/k_j)^{k_j}$$

Time-dependent Hamiltonian simulation

$$\frac{d}{dt}u(t) = -iH(t)u(t), \quad 0 \leq t \leq T, \quad H(t) \text{ is Hermitian}$$

- Product formula: commutator scaling (including derivatives), suboptimal in T and ϵ
- Truncated Dyson series method: near-optimal time and precision dependence

$$\mathcal{O}\left(\max \|H(t)\| T \ \log\left(\frac{\max \|H(t)\| T}{\epsilon}\right)\right)$$

Reference: [arXiv:2411.03180]

Application: Linear systems of equations

Quantum linear system problem (QLSP)

Classical: given A: $N \times N$ Hermitian matrix, b: N-dimensional vector, compute

$$x = A^{-1}b$$

- Non-Hermitian: consider $\begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix} \begin{pmatrix} 0 \\ x \end{pmatrix} = \begin{pmatrix} b \\ 0 \end{pmatrix}$

Quantum: find an ϵ-approximation of the quantum state

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|}$$

- Assume $\|A\| = 1$ and we have black-box access to A and $|b\rangle$

Important parameters: tolerated error level ϵ, condition number $\kappa = \|A\|\|A^{-1}\|$

QSVT for QLSP

$$A = W\Sigma V^\dagger \quad \rightarrow \quad A^{-1} = V\Sigma^{-1}W^\dagger$$

Consider the function

$$f(x) = \frac{1}{\kappa x}$$

Polynomial approximation: can find an odd polynomial $p(x)$ of degree $\mathcal{O}(\kappa \log(1/\epsilon))$ such that $|p(x)| \leq 1$ and

$$|p(x) - 1/(\kappa x)| \leq \epsilon, \quad \forall\ x \in [1/\kappa, 1]$$

Algorithm: use QSVT to implement $p(A^\dagger)$, apply to $|0\rangle\,|b\rangle$, and post-select

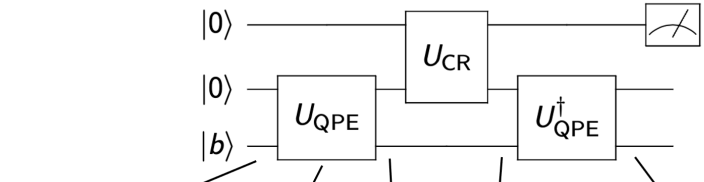
Harrow-Hassidim-Lloyd (HHL)

- The first quantum algorithm for solving QLSP
- Key equation: let $(\lambda_j, |v_j\rangle)$ be the eigenvalues and eigenvectors of A, and $|b\rangle = \sum_{j=0}^{N-1} \beta_j |v_j\rangle$, then

$$A^{-1}|b\rangle = \left(\sum_{j=0}^{N-1} \lambda_j^{-1} |v_j\rangle \langle v_j|\right) \left(\sum_{j=0}^{N-1} \beta_j |v_j\rangle\right) = \sum_{j=0}^{N-1} \frac{\beta_j}{\lambda_j} |v_j\rangle$$

- Subroutines:
  - Quantum phase estimation (QPE): store the information (binary encoding) of λ_j's in an ancilla register coherently
  - Controlled rotation: multiply the factor λ_j^{-1} to each eigenvector $|v_j\rangle$

HHL



$\sum_j \beta_j |0\rangle |0\rangle |v_j\rangle$

QPE for $e^{2\pi i A}$

$\sum_j \beta_j |0\rangle |\lambda_j\rangle |v_j\rangle$

$\sum_j \beta_j \left(\frac{C}{\lambda_j} |0\rangle + \sqrt{1 - \frac{C^2}{\lambda_j^2}} |1\rangle \right) |\lambda_j\rangle |v_j\rangle$

$= \sum_j C \frac{\beta_j}{\lambda_j} |0\rangle |\lambda_j\rangle |v_j\rangle + \sum_j * |1\rangle |\lambda_j\rangle |v_j\rangle$

$\sum_j C \frac{\beta_j}{\lambda_j} |0\rangle |0\rangle |v_j\rangle + \sum_j * |1\rangle |0\rangle |v_j\rangle$

$C \sim 1/\kappa$

LCU: Fourier approach

- Key identity:

$$\frac{1}{x} = \frac{i}{\sqrt{2\pi}} \int_0^\infty dy \int_{-\infty}^\infty dz z e^{-z^2/2} e^{-ixyz}.$$

- For a Hermitian matrix A,

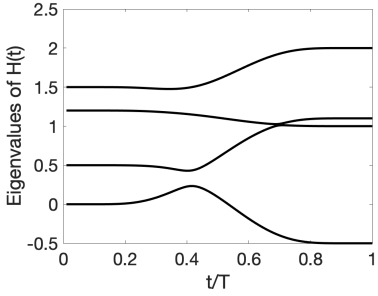
$$\begin{aligned} A^{-1} &= \frac{i}{\sqrt{2\pi}} \int_0^\infty dy \int_{-\infty}^\infty dz z e^{-z^2/2} e^{-iyzA} \\ &\approx \frac{i}{\sqrt{2\pi}} \int_0^Y dy \int_{-Z}^Z dz z e^{-z^2/2} e^{-iyzA} \\ &\approx \sum c_{j,j'} e^{-iy_j z_{j'} A} \end{aligned}$$

- Implemented by LCU + Hamiltonian simulation

Adiabatic Quantum Computing (AQC)

$$\imath\partial_t |\psi(t)\rangle = H(t/T)\,|\psi(t)\rangle\,, \quad t \in [0, T]$$
$$H(0)\,|\psi(0)\rangle = \lambda_0\,|\psi(0)\rangle$$

- Eigenvector of $H(0) \rightarrow$ Eigenvector of $H(1)$, if
  - Hamiltonian is slow enough (equivalently T is large enough)
  - gap condition
- Application: a quantum computing model to solve eigenvalue problem



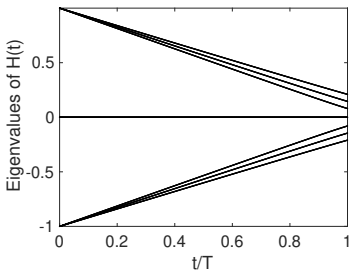
AQC for QLSP

AQC for QLSP algorithm:

$$H_0 = \begin{pmatrix} 0 & Q_b \\ Q_b & 0 \end{pmatrix}, \quad H_1 = \begin{pmatrix} 0 & AQ_b \\ Q_bA & 0 \end{pmatrix},$$

$$Q_b = I - |b\rangle \langle b|$$

$$H(s) = (1 - f(s))H_0 + f(s)H_1,$$



- Eigenpath corresponding to eigenvalue 0 is of interest, which connects $[b; 0]$ and $[x; 0]$
- Need to carefully choose the scheduling function $f(s)$

Complexities

Methods	Complexity	
	A	b
HHL[arXiv:0811.3171]	$\mathcal{O}(\kappa^2/\epsilon)$	$\mathcal{O}(\kappa)$
LCU[arXiv:1511.02306]	$\mathcal{O}(\kappa^2 \text{ poly}\log(\kappa/\epsilon))$	$\mathcal{O}(\kappa)$
QSVT[arXiv:1804.01973]	$\mathcal{O}(\kappa^2 \log(\kappa/\epsilon))$	$\mathcal{O}(\kappa)$
RM[arXiv:1805.10549]	$\mathcal{O}((\kappa/\epsilon)\log(\kappa))$	$\mathcal{O}((\kappa/\epsilon)\log(\kappa))$
AQC(p)[arXiv:1909.05500]	$\mathcal{O}((\kappa/\epsilon)\log(\kappa/\epsilon))$	$\mathcal{O}((\kappa/\epsilon)\log(\kappa/\epsilon))$
AQC(exp)[arXiv:1909.05500]	$\mathcal{O}(\kappa \text{ poly}\log(\kappa/\epsilon))$	$\mathcal{O}(\kappa \text{ poly}\log(\kappa/\epsilon))$
dAQC+EF[arXiv:2111.08152]	$\mathcal{O}(\kappa\log(1/\epsilon))$	$\mathcal{O}(\kappa\log(1/\epsilon))$
KR[arXiv:2406.12086]	$\mathcal{O}(\kappa\log(1/\epsilon))$	$\mathcal{O}(\kappa\log(1/\epsilon))$
Tunable VTAA[arXiv:2410.18178]	$\mathcal{O}(\kappa\log(\kappa/\epsilon)\log(1/\epsilon))$	$\mathcal{O}(\kappa)$
$\vdots$	$\vdots$	$\vdots$
Lower bound	$\Omega(\kappa\log(1/\epsilon))$	$\Omega(\kappa)$

Table: $\kappa = \|A\|\|A^{-1}\|$: condition number. ϵ: error

Summary

Summary

Quantum computing: use the nature of quantum mechanics to perform computing

Input models:

- Vector: quantum state
- Matrix: block-encoding

Algorithms:

- LCU
- Matrix functions: QSVT can implement polynomials of degree d with cost $\mathcal{O}(d)$
- Hamiltonian simulation: QSVT, product formula
- Linear systems of equations: QSVT, HHL, LCU, adiabatic

Additional resources

- IPAM tutorial: Mathematical and Computational Challenges in Quantum Computing Tutorials (Sept. 12-15, 2023)
  - `https://www.ipam.ucla.edu/programs/workshops/mathematical-and-computational-challenges-in-quantum-computing-tutorials/`
- Lecture notes by Lin Lin:
  - `https://math.berkeley.edu/~linlin/qasc/`

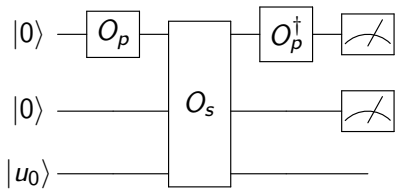
Thank you!

dongan@pku.edu.cn

Matrix addition

- Goal: compute (block encode) $\sum_j c_j A_j$ for general matrices $\|A_j\| \leq 1$
- Algorithm: LCU where unitaries are block-encodings

$$O_s = \sum_j |j\rangle\langle j| \otimes U_{A_j}, \qquad U_{A_j} = \begin{pmatrix} A_j & * \\ * & * \end{pmatrix}$$



Vector addition

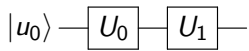
- Goal: compute $\sum_j c_j |u_j\rangle$ for quantum states $|u_j\rangle$
- Algorithm: Implement $\sum_j c_j U_j$ on $|0\rangle$ where U_j's are state preparation oracles

$$U_j : |0\rangle \to |u_j\rangle, \qquad \sum_j c_j U_j |0\rangle = \sum_j c_j |u_j\rangle$$

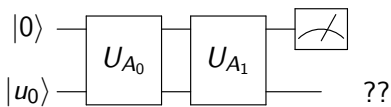
Matrix multiplication

Let us start with two matrices

- Two unitaries $U_1 U_0$



- Two matrices: A_0 and A_1, block-encodings U_{A_0} and U_{A_1}. Can we try this?



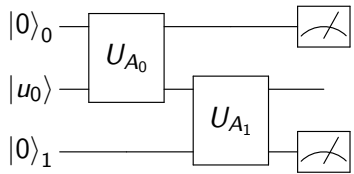
- Does not work:

$$U_{A_1} U_{A_0} |0\rangle |u_0\rangle = U_{A_1} \left(|0\rangle A_0 |u_0\rangle + |1\rangle |*\rangle \right)$$

$$\begin{pmatrix} A_1 & * \\ * & * \end{pmatrix} \begin{pmatrix} A_0 & * \\ * & * \end{pmatrix} \begin{pmatrix} u_0 \\ 0 \end{pmatrix} = \begin{pmatrix} A_1 & * \\ * & * \end{pmatrix} \begin{pmatrix} A_0 u_0 \\ * \end{pmatrix}$$

Matrix multiplication

Method 1: duplicate ancilla qubits

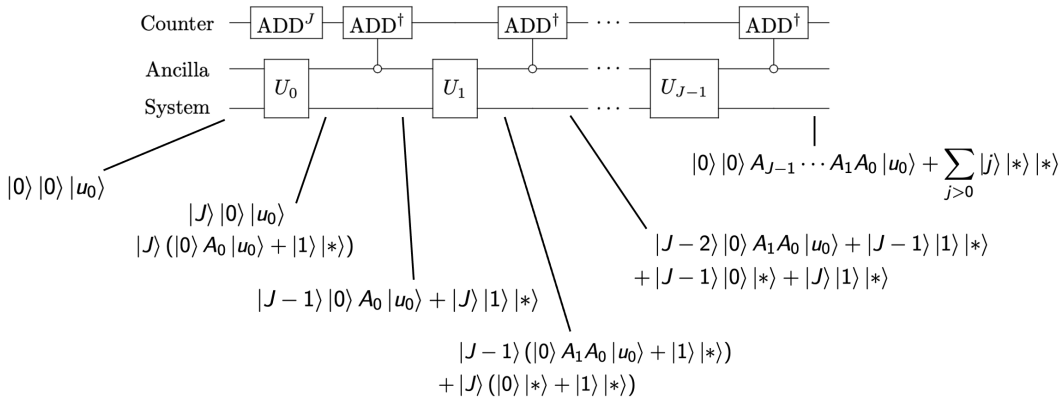


$$
\begin{aligned}
|0\rangle_1 |0\rangle_0 |u_0\rangle &\xrightarrow{U_{A_0}} |0\rangle_1 \left(|0\rangle_0 A_0 |u_0\rangle + |1\rangle_0 |*\rangle\right) \\
&= |0\rangle_0 |0\rangle_1 A_0 |u_0\rangle + |1\rangle_0 |0\rangle_1 |*\rangle \\
&\xrightarrow{U_{A_1}} |0\rangle_0 \left(|0\rangle_1 A_1 A_0 |u_0\rangle + |1\rangle_1 |*\rangle\right) + |1\rangle_0 |0\rangle_1 |*\rangle + |1\rangle_0 |1\rangle_1 |*\rangle
\end{aligned}
$$

Multiplication of J matrices: using $\mathcal{O}(J)$ extra ancilla qubits

Matrix multiplication

Method 2: compression gadget (Low-Wiebe [arXiv:1805.00675], Fang-Lin-Tong [arXiv:2208.06941])



$$\mathrm{ADD} : |0\rangle \to |1\rangle \to |2\rangle \to \cdots \to |J\rangle \to |0\rangle$$

Qubitization

So far we have assumed *Hermitian* block-encoding for Hermitian matrices (i.e., $U_A = U_A^\dagger$), now we relax this assumption

$$U_A |0\rangle |v_j\rangle = \lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\,|\perp_j'\rangle$$

where $\Pi |\perp_j'\rangle = 0$, $\Pi = |0\rangle\langle 0| \otimes I$.
Notice that since A is Hermitian,

$$U_A^\dagger = \begin{pmatrix} A & * \\ * & * \end{pmatrix}, \quad U_A^\dagger |0\rangle |v_j\rangle = \lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\,|\perp_j\rangle$$

where $\Pi |\perp_j\rangle = 0$. Apply U_A, then we have

$$|0\rangle |v_j\rangle = \lambda_j(\lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\,|\perp_j'\rangle) + \sqrt{1-\lambda_j^2}\,U_A |\perp_j\rangle$$

$$U_A |\perp_j\rangle = \sqrt{1-\lambda_j^2}\,|0\rangle |v_j\rangle - \lambda_j |\perp_j'\rangle$$

Qubitization

$$U_A |0\rangle |v_j\rangle = \lambda_j |0\rangle |v_j\rangle + \sqrt{1-\lambda_j^2}\, |\perp_j'\rangle$$
$$U_A |\perp_j\rangle = \sqrt{1-\lambda_j^2}\, |0\rangle |v_j\rangle - \lambda_j |\perp_j'\rangle$$

So U_A maps $\mathcal{H}_j = \text{span}\{|0\rangle |v_j\rangle, |\perp_j\rangle\}$ to $\mathcal{H}_j' = \text{span}\left\{|0\rangle |v_j\rangle, |\perp_j'\rangle\right\}$, we can also verify that $U_A^\dagger$ maps $\mathcal{H}_j'$ to $\mathcal{H}_j$,

$$[U_A]_{\mathcal{H}_j \to \mathcal{H}_j'} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}, \quad [U_A^\dagger]_{\mathcal{H}_j' \to \mathcal{H}_j} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}$$

For the projector $\Pi = |0\rangle\langle 0| \otimes I$, $Z_\Pi = 2\Pi - 1$,

$$[Z_\Pi]_{\mathcal{H}_j} = [Z_\Pi]_{\mathcal{H}_j'} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Qubitization

Let $\mathcal{H}_j = \text{span}\{|0\rangle|v_j\rangle, |\perp_j\rangle\}$, $\mathcal{H}'_j = \text{span}\left\{|0\rangle|v_j\rangle, |\perp'_j\rangle\right\}$,

$$[U_A]_{\mathcal{H}_j \to \mathcal{H}'_j} = [U_A^\dagger]_{\mathcal{H}'_j \to \mathcal{H}_j} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}, \quad [Z_\Pi]_{\mathcal{H}_j} = [Z_\Pi]_{\mathcal{H}'_j} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Then

$$[U_A^\dagger Z_\Pi U_A Z_\Pi]_{\mathcal{H}_j} = \begin{pmatrix} \lambda_j & \sqrt{1-\lambda_j^2} \\ \sqrt{1-\lambda_j^2} & -\lambda_j \end{pmatrix}^2, \quad [(U_A^\dagger Z_\Pi U_A Z_\Pi)^k]_{\mathcal{H}_j} = \begin{pmatrix} T_{2k}(\lambda_j) & * \\ * & * \end{pmatrix}.$$

Therefore $(U_A^\dagger Z_\Pi U_A Z_\Pi)^k$ block encodes $T_{2k}(A)$

$$(U_A^\dagger Z_\Pi U_A Z_\Pi)^k = \begin{pmatrix} T_{2k}(A) & * \\ * & * \end{pmatrix}$$

Qubitization

For odd polynomials, notice that $\mathcal{H}_j$ and $\mathcal{H}_j'$ share common $|0\rangle|v_j\rangle$,

$$[U_A Z_\Pi (U_A^\dagger Z_\Pi U_A Z_\Pi)^k]_{\mathcal{H}_j \to \mathcal{H}_j'} = \begin{pmatrix} T_{2k+1}(\lambda_j) & * \\ * & * \end{pmatrix},$$

$$U_A Z_\Pi (U_A^\dagger Z_\Pi U_A Z_\Pi)^k = \begin{pmatrix} T_{2k+1}(A) & * \\ * & * \end{pmatrix}$$

Quantum signal processing (QSP)

$$U_\Phi(\lambda) = e^{i\phi_0 Z} \prod_{j=1}^{d} \left[U(\lambda) Z e^{i\phi_j Z} \right] = \begin{pmatrix} P(\lambda) & -Q(\lambda)\sqrt{1-\lambda^2} \\ Q^*(\lambda)\sqrt{1-\lambda^2} & P^*(\lambda) \end{pmatrix}$$

Use $Ze^{i\phi Z} = (-i)e^{i(\phi+\pi/2)Z} = ie^{i(\phi-\pi/2)Z}$, we can combine Z and $e^{i\phi Z}$ with modified phase factors:

- If d is even, then $\widetilde{\phi}_j = \phi_j + (-1)^j \pi/2, j \neq 0$, $\widetilde{\phi}_0 = \phi_0$
- If d is odd, then $\widetilde{\phi}_j = \phi_j + (-1)^j \pi/2, j \neq 0$. We will have an extra Z, but this does not affect the bottom-left element

$$e^{i\widetilde{\phi}_0 Z} \prod_{j=1}^{d} \left[U(\lambda) e^{i\widetilde{\phi}_j Z} \right] = \begin{pmatrix} P(\lambda) & * \\ * & * \end{pmatrix}$$

QSP + Qubitization

Consider Hermitian matrix A and a polynomial $P(x)$ satisfying QSP conditions. According to qubitization,

- If d is even:

$$e^{i\widetilde{\phi}_0 Z_\Pi} \prod_{j=1}^{d/2} \left[U_A^\dagger e^{i\widetilde{\phi}_{2j-1} Z_\Pi} U_A e^{i\widetilde{\phi}_{2j} Z_\Pi} \right] = \begin{pmatrix} P(A) & * \\ * & * \end{pmatrix}$$

- If d is odd:

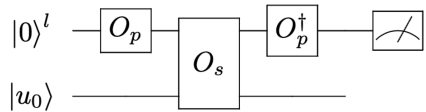
$$e^{i\widetilde{\phi}_0 Z_\Pi} U_A e^{i\widetilde{\phi}_1 Z_\Pi} \prod_{j=1}^{(d-1)/2} \left[U_A^\dagger e^{i\widetilde{\phi}_{2j} Z_\Pi} U_A e^{i\widetilde{\phi}_{2j+1} Z_\Pi} \right] = \begin{pmatrix} P(A) & * \\ * & * \end{pmatrix}$$

QSP/QSVT vs LCU

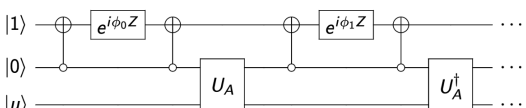
- Both QSP/QSVT and LCU can implement matrix functions
- For Hermitian matrices, LCU has computational overhead due to 1-norm of the coefficients and require extra control logic
- For general matrices
  - QSVT: singular value transformation
  - LCU: eigenvalue transformation

$$f(A) = \frac{1}{2\pi i} \int_\Gamma f(z)(z - A)^{-1} dz.$$

LCU:

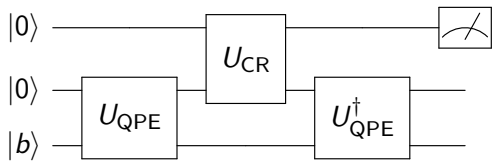


QSP:



HHL

- Complexity analysis: two sources
  - The cost of a single run: mainly due to QPE
    $$\mathcal{O}(\kappa/\epsilon)$$
  - Number of repeats:
    $$\mathcal{O}(1/(C\|A^{-1}|b\rangle\|)) = \mathcal{O}(\kappa)$$
- Overall complexity:
  $$\mathcal{O}(\kappa^2/\epsilon)$$
- Need two efficient subroutines:
  - Hamiltonian simulation: implement $e^{2\pi i A}$
  - QPE



Output (here $C \sim 1/\kappa$):

$$\sum_j C\,|0\rangle\,|0\rangle\,(A^{-1}\,|b\rangle) + |\perp\rangle$$

LCU: Chebyshev approach

- Idea: expand $1/x$ using Chebyshev polynomials
  - Chebyshev polynomials:

$$T_n(\cos(\theta)) = \cos(n\theta)$$
$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \quad T_0(x) = 1, T_1(x) = x$$

  - Bounded by 1 on $[-1, 1]$, minimize Runge's phenomenon, close to the best polynomial approximation
- Approach:

$$\begin{aligned}
\frac{1}{x} &\approx \frac{1-(1-x^2)^d}{x} && (d \sim \kappa^2 \log(\kappa/\epsilon)) \\
&= 4\sum_{j=0}^{d-1}(-1)^j \left(2^{-2d}\sum_{i=j+1}^{d}\binom{2d}{d+i}\right) T_{2j+1}(x) \\
&\approx 4\sum_{j=0}^{J}(-1)^j \left(2^{-2d}\sum_{i=j+1}^{d}\binom{2d}{d+i}\right) T_{2j+1}(x) && (J \sim \sqrt{d\log(d/\epsilon)})
\end{aligned}$$

LCU: Chebyshev approach

$$A^{-1} \approx \sum_{j=0}^{\mathcal{O}(\kappa \text{ poly}\log(\kappa/\epsilon))} c_j T_{2j+1}(A)$$

- $\| T_{2j+1}(A) \| \leq 1$ but not unitary, so we need to construct its block-encoding
- The same overall complexity:

$$\kappa \text{ poly}\log(\kappa/\epsilon) \times \kappa\sqrt{\log(\kappa/\epsilon)} = \mathcal{O}(\kappa^2 \text{poly}\log(\kappa/\epsilon))$$

Harrow-Hassidim-Lloyd (HHL)

- [5]The first quantum algorithm for solving QLSP
- Key equation: let $(\lambda_j, |v_j\rangle)$ be the eigenvalues and eigenvectors of A, and $|b\rangle = \sum_{j=0}^{N-1} \beta_j |v_j\rangle$, then

$$A^{-1}|b\rangle = \left(\sum_{j=0}^{N-1} \lambda_j^{-1} |v_j\rangle \langle v_j|\right)\left(\sum_{j=0}^{N-1} \beta_j |v_j\rangle\right) = \sum_{j=0}^{N-1} \frac{\beta_j}{\lambda_j} |v_j\rangle$$

- Need to do:
  - store the information (binary encoding) of λ_j's in an ancilla register coherently
  - multiply the factor λ_j^{-1} to each eigenvector $|v_j\rangle$

[5][arXiv:0811.3171]

HHL

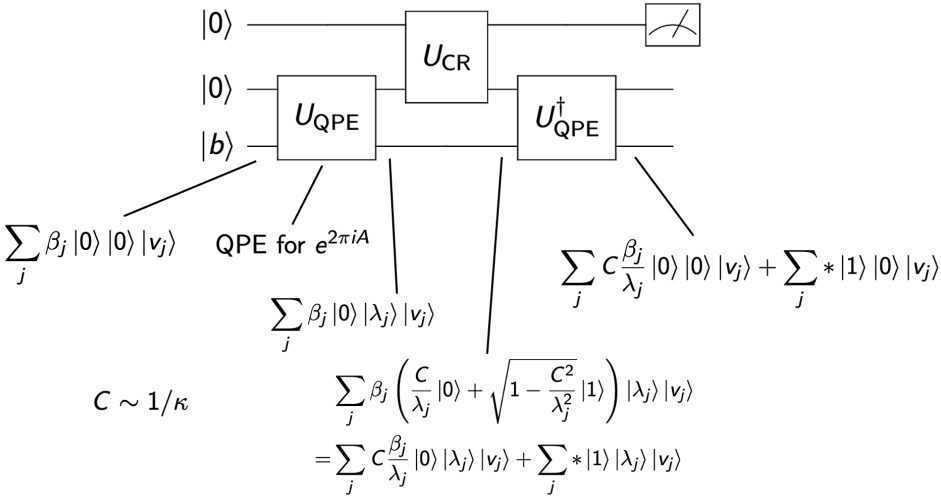
- Need to do:
  - store the information (binary encoding) of λ_j's in an ancilla register coherently
  - multiply the factor λ_j^{-1} to each eigenvector $|v_j\rangle$
- Useful subroutines:
  - Quantum phase estimation (QPE): Let U be a unitary and $U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$ for a real number $\theta \in [0,1]$. The (ideal) QPE algorithm is U_{QPE} such that

$$U_{\text{QPE}}|\psi\rangle|0\rangle = |\psi\rangle|\theta\rangle.$$

    Here $|\theta\rangle = |\theta_{m-1}\rangle \cdots |\theta_1\rangle|\theta_0\rangle$ where $\theta = (.\theta_0\theta_1\cdots\theta_{m-1})$ is its binary representation
  - Controlled rotation: A unitary U_{CR} such that

$$U_{\text{CR}}|\theta\rangle|0\rangle = |\theta\rangle\left(f(\theta)|0\rangle + \sqrt{1-|f(\theta)|^2}|1\rangle\right)$$

HHL


$|0\rangle$
$|0\rangle$
$|b\rangle$
U_{QPE}
U_{CR}
$U_{\mathrm{QPE}}^{\dagger}$
$\sum_j \beta_j |0\rangle |0\rangle |v_j\rangle$
QPE for $e^{2\pi i A}$
$\sum_j \beta_j |0\rangle |\lambda_j\rangle |v_j\rangle$
$\sum_j \beta_j \left(\frac{C}{\lambda_j}|0\rangle + \sqrt{1-\frac{C^2}{\lambda_j^2}}|1\rangle\right)|\lambda_j\rangle |v_j\rangle$
$= \sum_j C\frac{\beta_j}{\lambda_j}|0\rangle |\lambda_j\rangle |v_j\rangle + \sum_j * |1\rangle |\lambda_j\rangle |v_j\rangle$
$\sum_j C\frac{\beta_j}{\lambda_j}|0\rangle |0\rangle |v_j\rangle + \sum_j * |1\rangle |0\rangle |v_j\rangle$
$C \sim 1/\kappa$

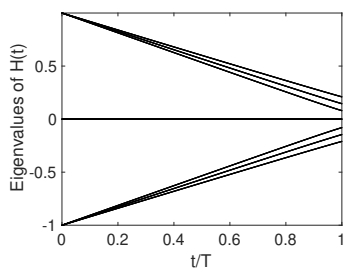
Quantum Adiabatic Theorem

Theorem (Jansen-Ruskai-Seiler (arXiv:quant-ph/0603175))

Assume gap $\Delta(s)$, then the distance between the dynamics and the eigenvector can be bounded by

$$\eta(s) = C\Big\{ \frac{\|H'(0)\|_2}{T\Delta^2(0)} + \frac{\|H'(s)\|_2}{T\Delta^2(s)} + \frac{1}{T}\int_0^s \left(\frac{\|H''(\tau)\|_2}{\Delta^2(\tau)} + \frac{\|H'(\tau)\|_2^2}{\Delta^3(\tau)} \right) d\tau \Big\}.$$

- To bound the error by ϵ: $T = \mathcal{O}(\Delta_*^{-3}\epsilon^{-1})$
- Cubic dependence on the gap
- In QLSP, $\Delta_* \sim 1/\kappa \Longrightarrow T = \mathcal{O}(\kappa^3\epsilon^{-1})$

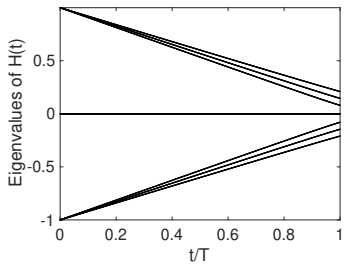


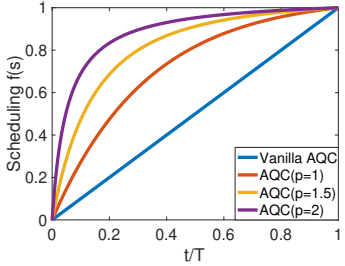
Time-optimal AQC

$$\imath\partial_t |\psi(t)\rangle = H(t/T)\,|\psi(t)\rangle\,, \quad t \in [0, T]$$

- Idea: generally interpolate $H(s) = (1 - f(s))H_0 + f(s)H_1$, choose proper $f(s)$ to slow down the Hamiltonian when the gap is small
- AQC(p): $\dot{f}(s) = c\Delta^p(f(s))$

$$\Longrightarrow \quad T = \mathcal{O}\left(\kappa/\epsilon\right) \quad [6]$$





[6] An-Lin [arXiv:1909.05500]

AQC(exp)

- Quantum adiabatic theorem can be improved to error $\sim \mathcal{O}(T^{-k})$ if we only care about the final state error[7]
- Requiring boundary cancellation condition, *i.e.*, the support of $H'(s)$ is in $(0,1)$

$$\eta(s) = C\Big\{\frac{\|H'(0)\|_2}{T\Delta^2(0)} + \frac{\|H'(s)\|_2}{T\Delta^2(s)} + \frac{1}{T}\int_0^s \left(\frac{\|H''(\tau)\|_2}{\Delta^2(\tau)} + \frac{\|H'(\tau)\|_2^2}{\Delta^3(\tau)}\right) d\tau\Big\}.$$

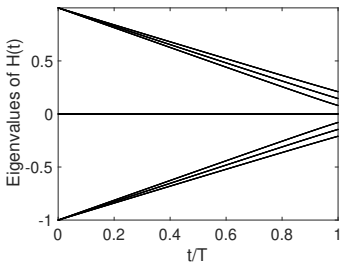
- $\text{Error} = \text{Boundary}_1 + \frac{1}{T}\int_0^1 = \text{Boundary}_1 + \text{Boundary}_2 + \frac{1}{T^2}\int_0^1 = \text{Boundary}_1 + \text{Boundary}_2 + \text{Boundary}_3 + \frac{1}{T^3}\int_0^1 = \cdots$

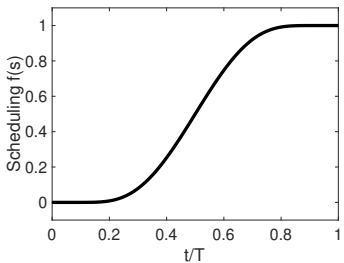
[7] Nenciu(1993)

AQC(exp)

- AQC(exp): $f(s) = c^{-1} \int_0^s \exp[-u^{-1}(1-u)^{-1}]du$, happens to be slow as well at the smallest gap

$$\implies T = \mathcal{O}\left(\kappa \text{ poly}\log(\kappa/\epsilon)\right) \quad [8]$$





[8] An-Lin [arXiv:1909.05500]

Preconditioning

- QLSP algorithms with cost $\mathcal{O}(\kappa \operatorname{poly}\log(\kappa/\epsilon))$, can still be expensive if the system is ill-conditioned
- Classical solution: preconditioning

$$Ax = b \iff MAx = Mb$$

- Effective if
  - $\kappa(MA) \ll \kappa(A)$
  - the matrix-vector multiplication My is easily accessible, and in particular its cost is independent of $\kappa(M)$
- Classical: diagonal matrix, incomplete factorization, sparse approximate inverse (SPAI), etc.
- Quantum:
  - SPAI (Clader-Jacobs-Sprouse [arXiv:1301.2340])
  - Circulant matrix (Shao-Xiang [arXiv:1807.04563])
  - Diagonal matrix (Tong-An-Wiebe-Lin [arXiv:2008.13295])

Preconditioning

$$(A + B)\,|x\rangle \sim |b\rangle$$

- Assume A is easily invertible with very large $\|A\|$ and moderate $\|B\|, \|A^{-1}\|, \|(A+B)^{-1}\|$
- $\kappa(A + B) \sim \mathcal{O}(\|A\|)$
- An example: Poisson's equation $-\Delta u(r) + V(r)u(r) = b(r)$
- Preconditioner: A^{-1}
  - $\kappa(I + A^{-1}B) = \mathcal{O}(1)$
- Algorithm:
  $A^{-1} \to A^{-1}B \to I + A^{-1}B \to (I + A^{-1}B)^{-1} \to (I + A^{-1}B)^{-1}A^{-1} = (A + B)^{-1}$
  - Need matrix addition and multiplication, and fast-inversion of a diagonal matrix